SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE **Duration: 2hrs**

Code of Conduct:

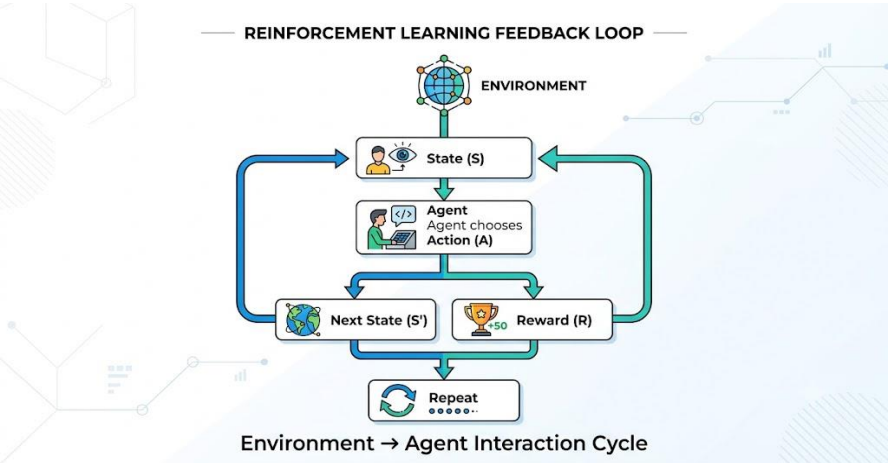
I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Jahnavi

1)Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Reinforcement Learning (RL) is a learning paradigm where an **agent** interacts with an **environment** to achieve a goal by maximizing cumulative rewards.

RL Framework



Problem Statement

Design a simple Reinforcement Learning (RL) framework that demonstrates the interaction between an agent and an environment. The agent observes the current state, performs an action, receives a reward, and aims to maximize cumulative rewards through repeated interactions.

Objective

- Understand the Reinforcement Learning framework.
- Demonstrate agent–environment interaction.
- Explain the role of states, actions, rewards, and cumulative rewards.

Theory

Reinforcement Learning (RL) is a machine learning technique in which an **agent** learns by interacting with an **environment**. At each step, the agent observes the current **state**, selects an **action**, receives a **reward**, and moves to the next state. The goal is to maximize the **cumulative reward**, which represents the total discounted reward received over time.

Algorithm (Process)

1. Initialize the environment and the starting state.
2. Define possible actions.
3. The agent selects an action.
4. The environment returns a reward and the next state.
5. Repeat until the goal state is reached.

6. Calculate the cumulative reward.

CODE:

```
states = ["Start", "Middle", "Goal"]
current_state = states[0]
total_reward = 0

print("Initial State:", current_state)

while current_state != "Goal":
    print("\nAction: Move Forward")

    if current_state == "Start":
        reward = 5
        current_state = "Middle"
    else:
        reward = 10
        current_state = "Goal"

    total_reward += reward

    print("Current State:", current_state)
    print("Reward:", reward)

print("\nGoal Reached")
print("Cumulative Reward:", total_reward)
```

Sample Output

Initial State: Start

Action: Move Forward
Current State: Middle
Reward: 5

Action: Move Forward
Current State: Goal
Reward: 10

Goal Reached
Cumulative Reward: 15

Result

The agent successfully reached the goal while maximizing the cumulative reward.

Conclusion

The RL framework allows an agent to learn optimal decisions through interaction with the environment by maximizing long-term cumulative rewards.

2)Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

A Markov Decision Process (MDP) provides a mathematical framework for solving sequential decision-making problems.

An MDP consists of the tuple:

$$(S, A, P, R, \gamma)$$

Problem Statement

Develop a simple program to represent a Markov Decision Process (MDP) using its fundamental components.

Objective

- Understand MDP components.
- Represent sequential decision-making mathematically.

Theory

A **Markov Decision Process (MDP)** models sequential decision-making using five components:

- **State Space (S)** – Possible situations.
- **Action Space (A)** – Available actions.
- **Transition Probability (P)** – Probability of moving to the next state.
- **Reward Function (R)** – Immediate feedback.
- **Discount Factor (γ)** – Importance of future rewards.

Algorithm (Process)

1. Define the states.
2. Define available actions.
3. Assign transition probability.
4. Assign rewards.
5. Set the discount factor.
6. Display all MDP components.

Python Code

```
states = ["Start", "Middle", "Goal"]
actions = ["Left", "Right"]
```

```
transition_probability = 0.9

reward = {
    "Start": -1,
    "Middle": 5,
    "Goal": 100
}

gamma = 0.9

print("States:", states)
print("Actions:", actions)
print("Transition Probability:", transition_probability)
print("Reward Function:", reward)
print("Discount Factor:", gamma)
```

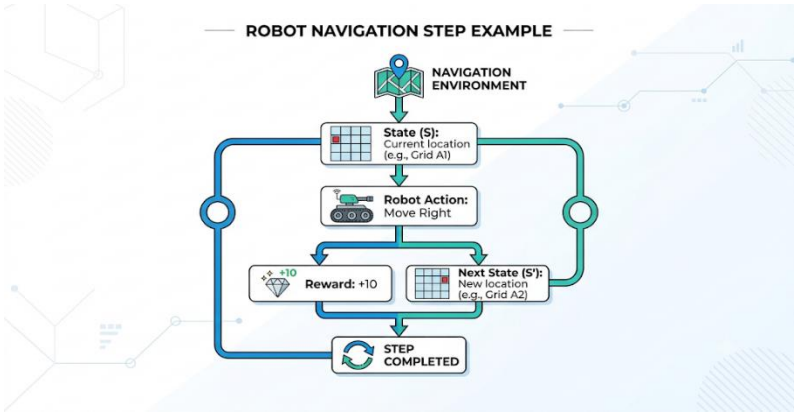
Sample Output

```
States: ['Start', 'Middle', 'Goal']
Actions: ['Left', 'Right']
Transition Probability: 0.9
Reward Function: {'Start': -1, 'Middle': 5, 'Goal': 100}
Discount Factor: 0.9
```

Result

The MDP model successfully represents states, actions, rewards, transition probability, and discount factor.

Example



Conclusion

MDPs provide a mathematical framework for solving sequential decision-making problems in Reinforcement Learning.

3)Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Problem Statement

Implement a simple Bellman Equation to compute the expected long-term value of a state.

Objective

- Understand policy and value functions.
- Compute state value using the Bellman Equation.

Theory

A **policy (π)** defines the action selected in each state.

A **state-value function V(s)** estimates the expected future reward of a state.

An **action-value function Q(s,a)** estimates the value of taking an action in a state.

Bellman Equation:

Algorithm (Process)

1. Initialize reward.
2. Set future state value.
3. Define discount factor.
4. Apply Bellman Equation.
5. Display the computed state value.

Python Code

```
reward = 5
future_value = 8
gamma = 0.9

state_value = reward + gamma * future_value

print("Reward:", reward)
print("Future Value:", future_value)
print("State Value:", state_value)
```

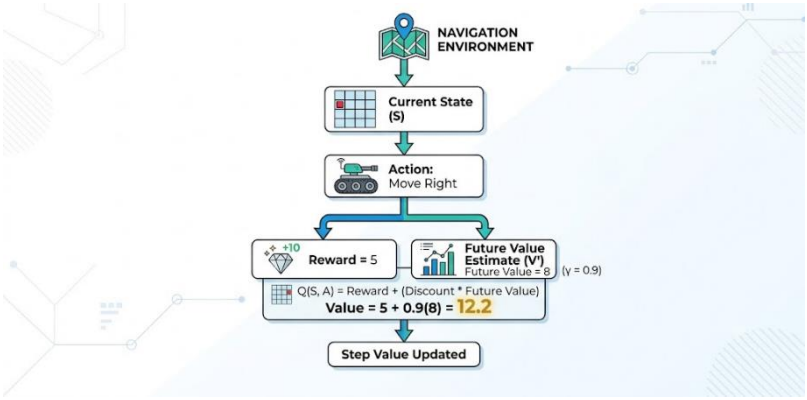
Sample Output

```
Reward: 5
Future Value: 8
State Value: 12.2
```

Result

The Bellman Equation successfully computes the expected long-term state value.

Example



Conclusion

Policy and value functions help an RL agent evaluate future rewards and select optimal actions.

4)Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Problem Statement

Develop a Python program to compare the characteristics of Model-Free and Model-Based Reinforcement Learning.

Objective

- Compare both RL approaches.
- Identify their strengths and limitations.

Theory

Model-Free RL

- Learns directly through interaction.
- Does not require an environment model.

Model-Based RL

- Uses a model of the environment.
- Performs planning before taking actions.

Algorithm (Process)

1. Store Model-Free RL characteristics.

2. Store Model-Based RL characteristics.
3. Display the comparison.

Python Code

```
model_free = {
   "Environment Model": "Not Required",
   "Learning Speed": "Slow",
   "Planning": "No"
}

model_based = {
   "Environment Model": "Required",
   "Learning Speed": "Fast",
   "Planning": "Yes"
}

print("Model-Free Reinforcement Learning")
for key, value in model_free.items():
   print(key, ":", value)

print("\nModel-Based Reinforcement Learning")
for key, value in model_based.items():
   print(key, ":", value)
```

Sample Output

```
Model-Free Reinforcement Learning
Environment Model : Not Required
Learning Speed : Slow
Planning : No

Model-Based Reinforcement Learning
Environment Model : Required
Learning Speed : Fast
Planning : Yes
```

Result

The comparison clearly shows the differences between Model-Free RL and Model-Based RL.

Comparison

Feature	Model-Free RL	Model-Based RL
Environment Model	Not Required	Required
Learning Speed	Slower	Faster
Planning	No	Yes
Sample Efficiency	Low	High

Conclusion

Model-Free RL is suitable for unknown environments, while Model-Based RL provides faster learning when an environment model is available.

5)Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

Q-learning is an off-policy reinforcement learning algorithm that learns the optimal action-value function.

Update Equation

$$\boldsymbol{Q(s, a) = Q(s, a) + \alpha[R + \gamma \max Q\,(s', a') - Q(s, a)]}$$

where

α = Learning Rate

γ = Discount Factor

Problem Statement

Implement the Q-learning update equation to calculate the updated Q-value after receiving a reward.

Objective

- Demonstrate the Q-learning update rule.
- Understand convergence behavior.

Theory

Q-learning is an **off-policy Reinforcement Learning algorithm** that updates the action-value function using:

where:

- α = Learning Rate
- γ = Discount Factor

Algorithm (Process)

1. Initialize Q-value.
2. Set reward.
3. Set future maximum Q-value.

4. Calculate the target value.
5. Update the Q-value.
6. Display the updated value.

Python Code

```
Q = 5
reward = 10
future_Q = 8

alpha = 0.5
gamma = 0.9

target = reward + gamma * future_Q

new_Q = Q + alpha * (target - Q)

print("Old Q-value:", Q)
print("Target:", target)
print("Updated Q-value:", new_Q)
```

Sample Output

Old Q-value: 5
Target: 17.2
Updated Q-value: 11.1

Result

The Q-value is successfully updated using the Q-learning equation, improving the agent's estimate of the optimal action.

Conclusion

Q-learning gradually updates action values through repeated interactions, enabling the agent to converge toward the optimal policy and maximize long-term rewards.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately